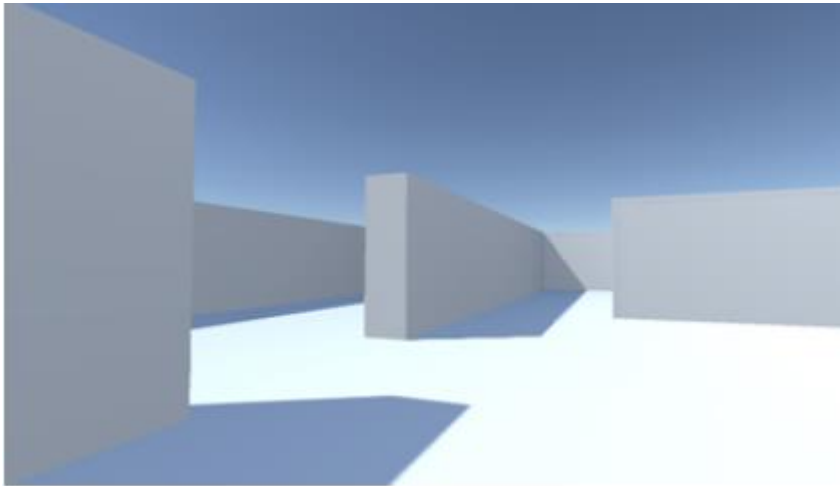


**PROGRAMOWANIE GIER
W UNITY I C#
WYKŁAD 4 –
OŚWIETLENIE, MATERIAŁY I TEXTURY,
SKYBOX, SYSTEM CZĄSTECZEK,
ZDARZENIA UNITY, RAYCASTING,
PIERWSZE GRY 3D,
NAWIGACJA AI**

Bogumiła Hnatkowska

WHITEBOXING

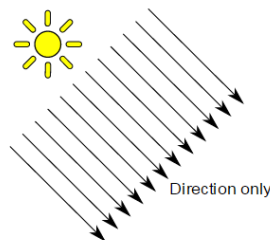
- Przygotowanie sceny z wykorzystaniem białych elementów (prostopadłościanów, sfer, ...) – pierwszy krok w projektowaniu gry 3D



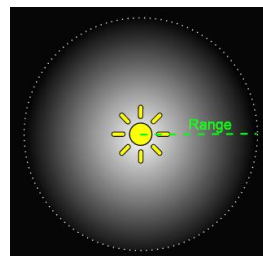
OŚWIETLENIE

○ Podstawowe typy źródeł światła:

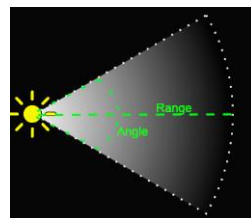
- Kierunkowe (directed)



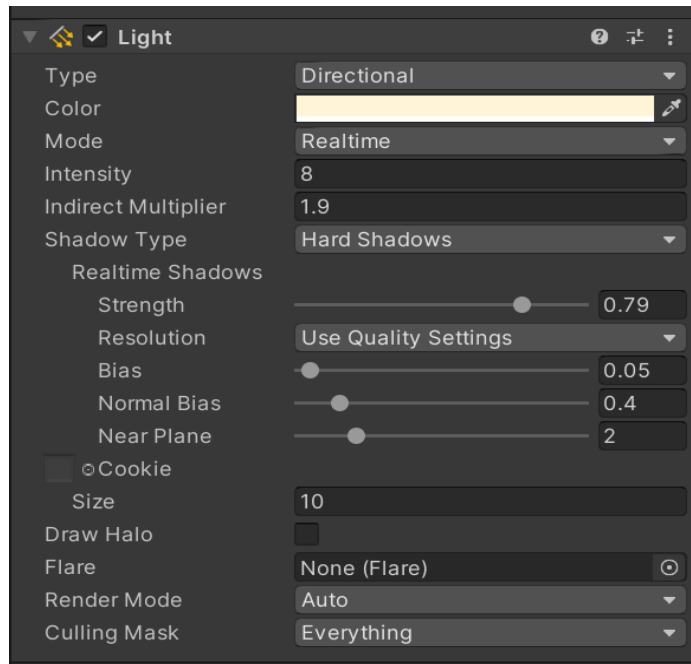
- Punktowe (point)



- Stożkowe /reflektorowe (spot)



OPIS PARAMETRÓW ŹRÓDŁA ŚWIATŁA



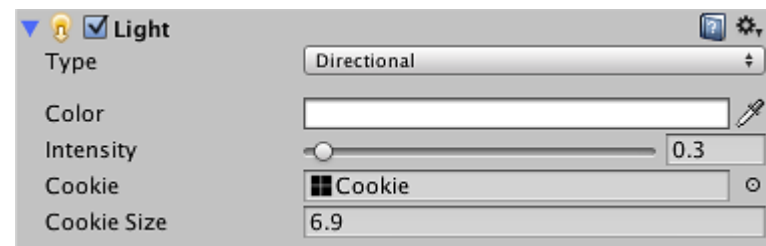
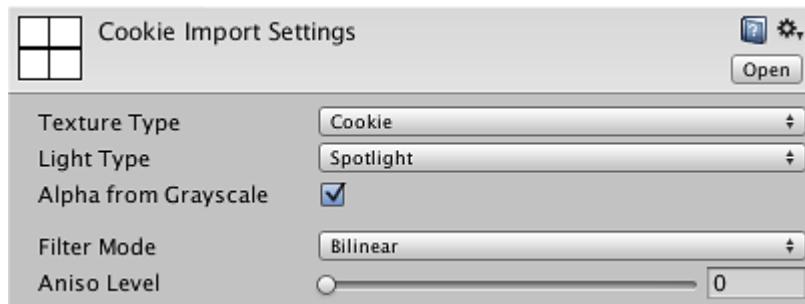
- Wybrane parametry:
 - Type
 - Color
 - Intensity
 - Shadow Type

Tworzenie źródła światła: GameObject > Light



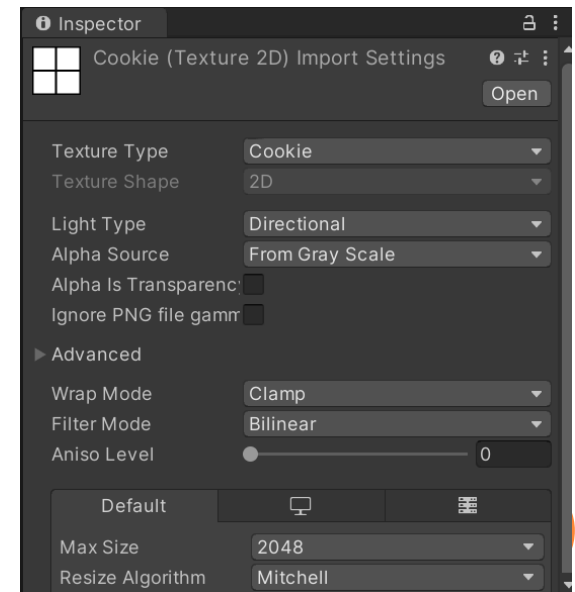
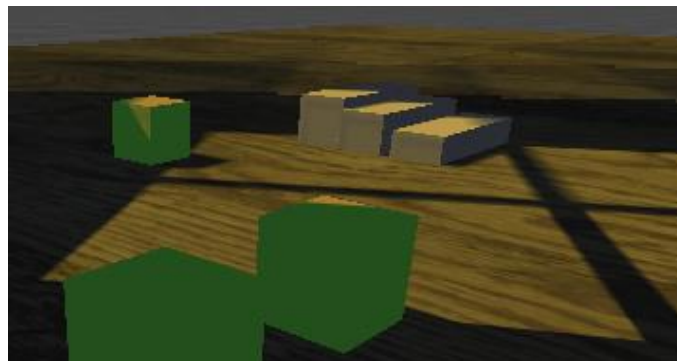
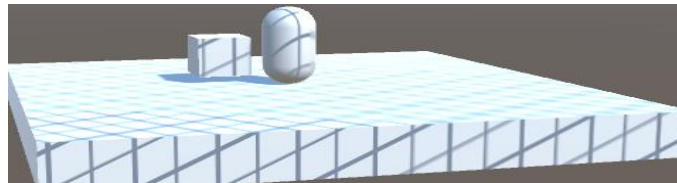
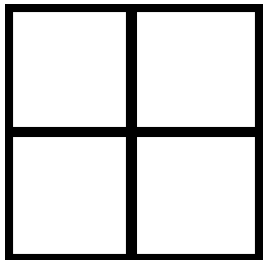
CIASZTECZKA (COOKIES)

- Cucoloris (cookie) – rodzaj filtra nałożonego na źródło światła, który wpływa na sposób emisji („kształt”) światła; może być użyte np. do symulowania „brudu” na drodze światła wpadającego przez okna
- Cookie – reprezentowane przez teksturę, w której ważny jest tylko kanał alfa; alternatywnie, można przygotować teksturę w różnych odcieniach szarości – przy zmianie jej typu na Cookie, można zaznaczyć opcję przekonwertowania odcienia szarości na kanał alfa; dla cookie należy odpowiednio ustawić typ światła



CIASZTECZKA (COOKIES), C.D.

- Światło punktowe: cookie typu PointLight, SpotLight
- Światło kierunkowe: cookie typu Directional, SpotLight
- Światło stożkowe: cookie typu SpotLight



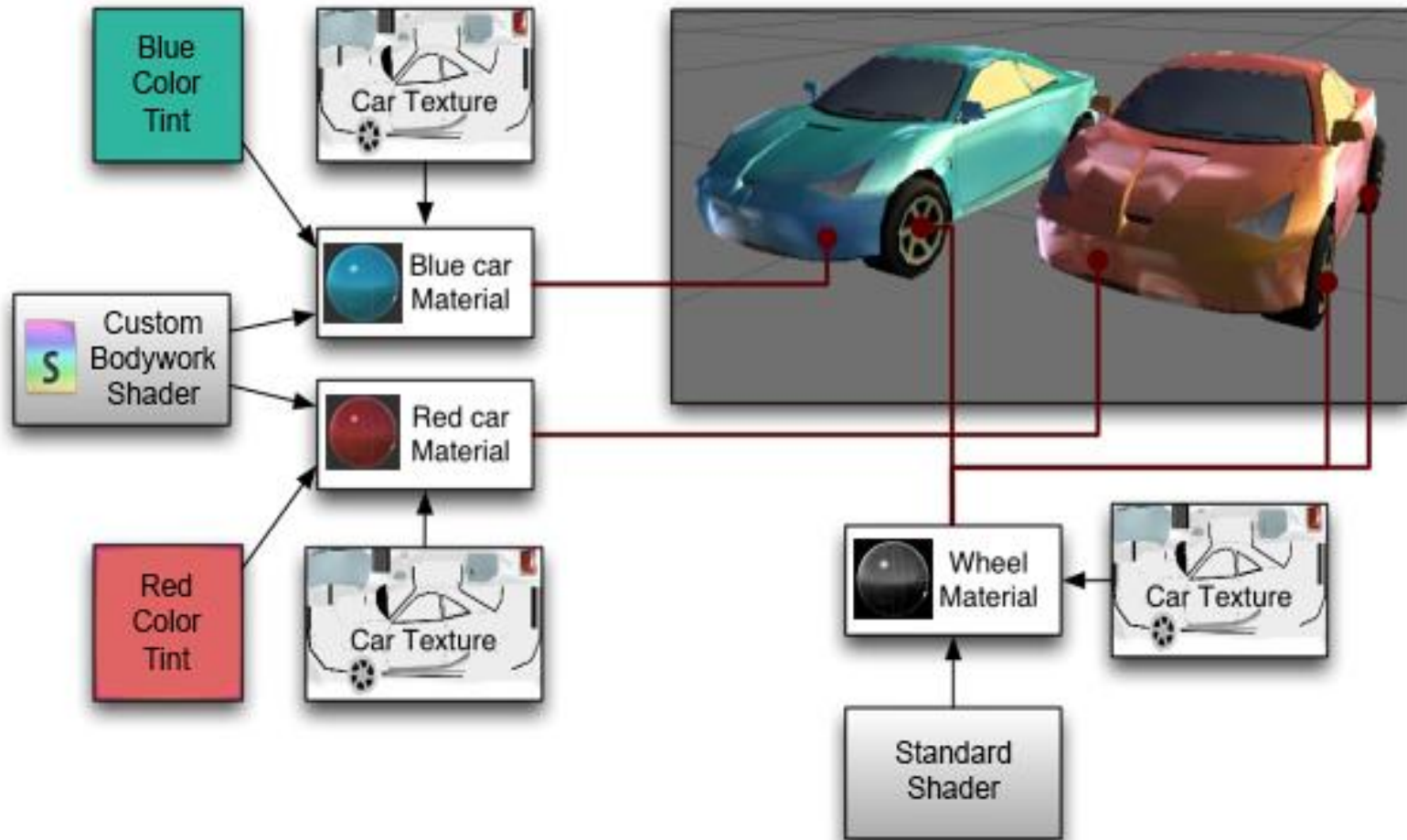
MATERIAŁY, TEKSUTRY I ZACIENIACZE

○ Renderowanie w Unity:

- Materiał (material) – zbiór informacji, które definiują własności powierzchni, w tym referencje do używanych tekstur, map opisujących typ powierzchni, czy kolor farby
- Tekstura (Texture) – obraz bitowy
- Zacieniacz (shader) – program napisany w specjalnym języku, zawierający algorytm obliczający kolor dla każdego piksela na bazie oświetlenia i konfiguracji materiału; w większości przypadków odpowiedni jest standardowy zacieniacz (Standard Shader)
- Standardowy zacieniacz został zaprojektowany dla twardych powierzchni i radzi sobie dobrze z większością materiałów, jak kamienie, szkło, ceramika, guma. Nieźle również z materiałami takimi jak skóra, włosy, ubranie.

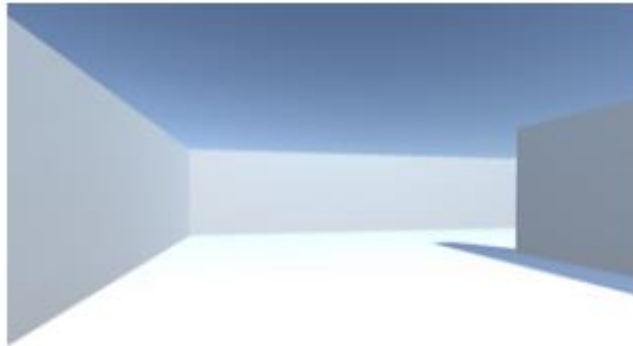


MATERIAŁY, TEKSUTRY I ZACIENIACZE, C.D.

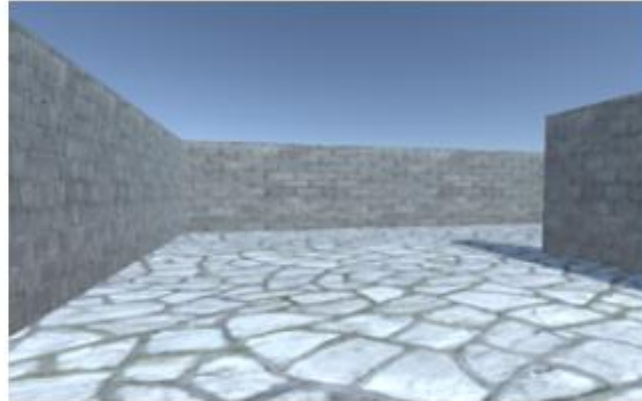


PRZYGOTOWANIE TEKSTUR

Before texturing
(only shading from lights)



After applying textures
(1 floor texture, 1 on all walls)



PRZYGOTOWANIE TEKSTUR, C.D.

- Wspierane formaty:
 - PNG – kompresja bezstratna, z kanałem alfa
 - JPG – kompresja stratna, bez kanału alfa
 - GIF – kompresja stratna, bez kanału alfa
 - BMP – domyślny dla Windows; bez kompresji; bez kanału alfa
 - TGA – z kanałem alfa; kompresja bezstratna (często wykorzystywany w grach)
 - TIFF – typowy dla zdjęć; bez kanału alfa; kompresja bezstratna
 - PICT – domyślny dla starszych Maców; bez kanału alfa; kompresja stratna
 - PSD – natywny format dla Photoshopa; bez kompresji; z kanałem alfa
- Kanał alfa jest używany do przechowywania informacji o przeźroczystości obrazu.



PRZYGOTOWANIE TEKSTUR, C.D.

- Obraz kafelkowy (tileable image) – obraz, dla którego wzory na przeciwnych brzegach są dopasowane (łatwo go wielokrotnie powielać).
- www.cgtextures.com

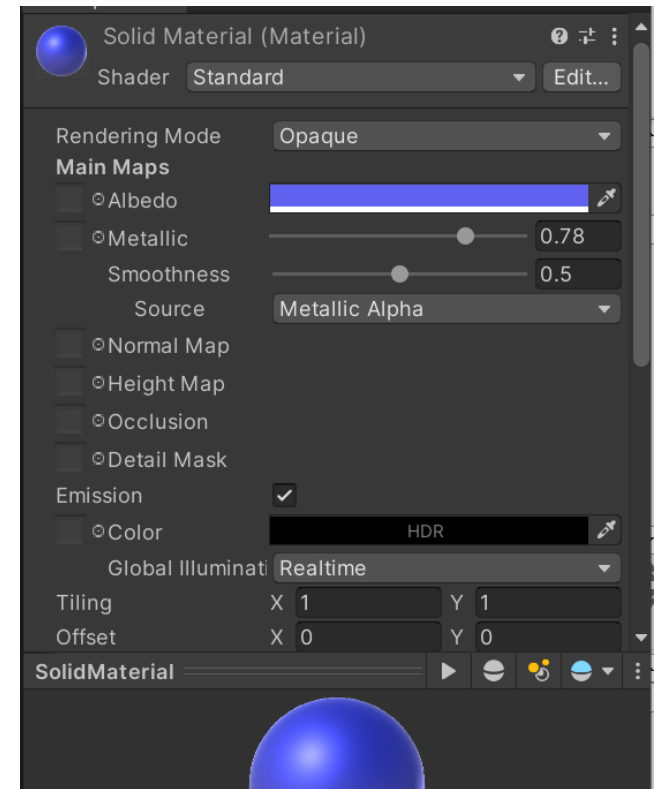
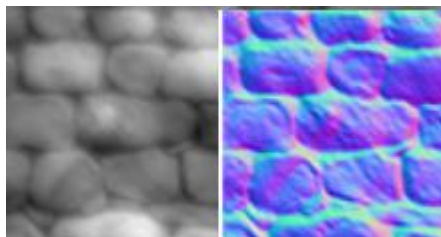


- Zalecane formaty: PNG, TGA
- Zalecane rozmiary: potęgi dwójki poniżej 4096, np. 256 x 256



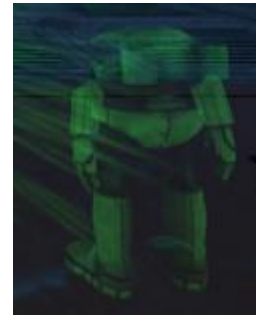
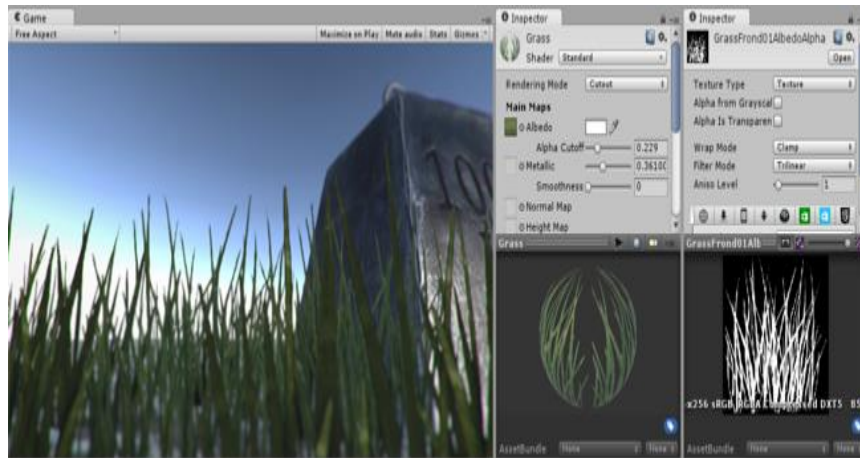
PRZYGOTOWANIE MATERIAŁÓW

- Tekstury są stosowane dla materiałów
- Tworzenie materiałów:
 - Przeciągnij teksturę na powierzchnię obiektu – materiał zostanie automatycznie utworzony
 - Assets > Create > Material; przeciągnij teksturę do slotu o nazwie Albedo (kolor bazowy), a następnie przeciągnij materiał na obiekt na scenie; dopasuj kafelkowanie (Tiling)
 - Możesz ustawić tekstury z mapami wysokości/normalnych

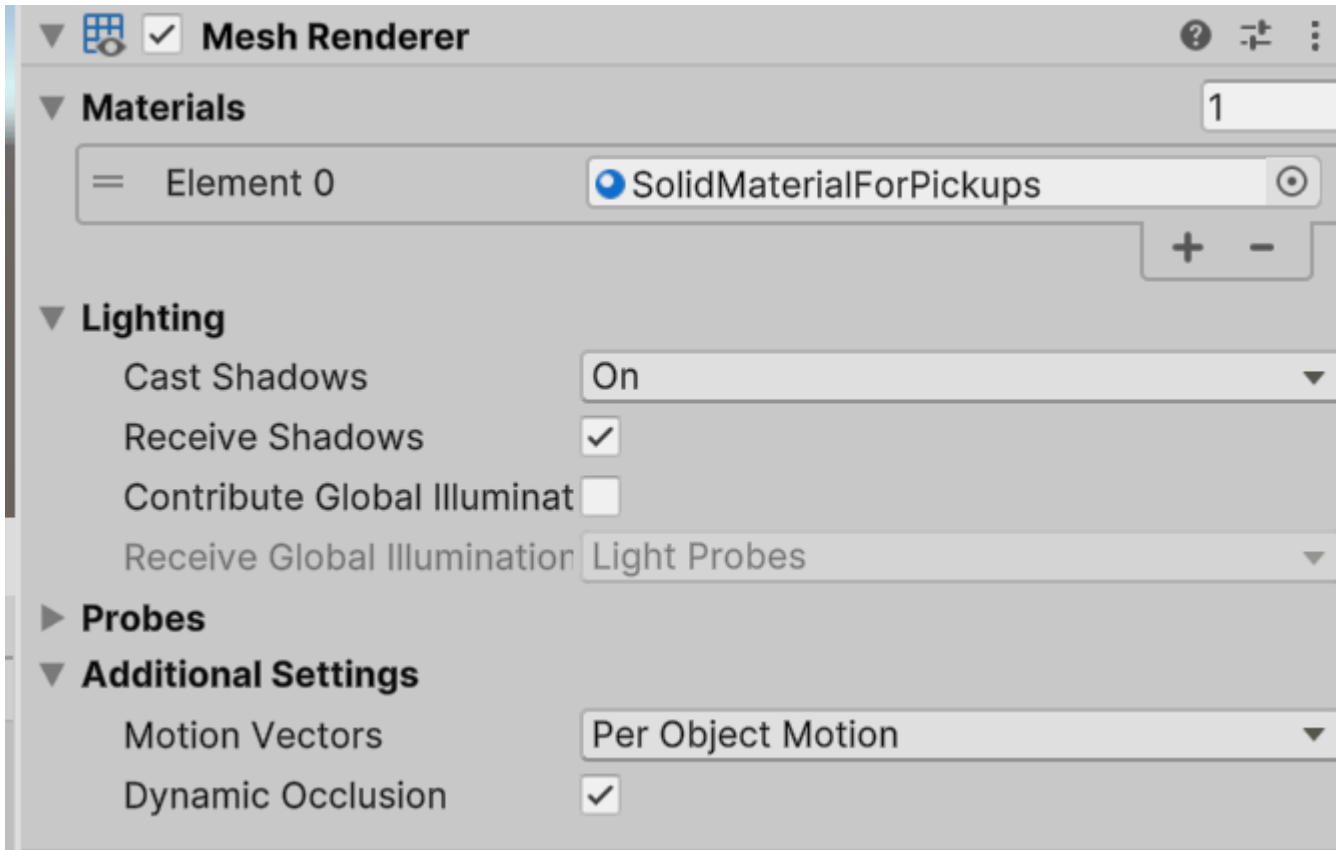


PRZYGOTOWANIE MATERIAŁÓW, WYBRANE PARAMETRY

- Tryb wyświetlania (Rendering Mode):
 - Nieprzezroczysty (Opaque)
 - Z progiem odcięcia (Cutout)
 - Przezroczysty (Transparent)
 - Wyblakły (Fade)



PARAMETRY MESH RENDERER



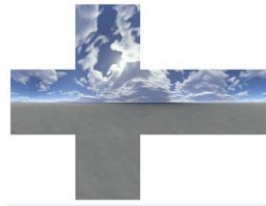
SKYBOX

- Skybox – sześciian otaczający kamerę wypełniony obrazami nieba na każdym boku (bez względu na to, w którą stronę kamera jest zwrócona, patrzy na obraz nieba)
- (Materiał) Skybox używa specjalnego zacięniacza;

- 6 stronny



- Cubemap



- Generowany proceduralnie
- Panoramiczny
- Window > Rendering > Lighting > Environment – ustawienie globalne (dla każdej kamery)
- Dodać komponent Skybox do kamery i ustawić w nim odpowiedni materiał (dla wybranej kamery)



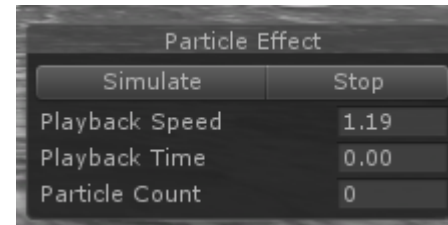
SYSTEM CZĄSTECZEK

- System cząsteczek (particle system) – mechanizm tworzenia i sterowania dużą liczbą poruszających się obiektów; użyteczny do tworzenia efektów wizualnych: ogień, dym, rozpryskująca się woda
- Cząsteczka (particle) – mały, prosty obrazek (image) lub siatka (mesh), które są tworzone i przemieszczane przez system cząsteczek; reprezentuje „porcję” obiektu; ma predefiniowany czas życia, w którym może się zmieniać (rotacja, wielkość, kolor)
- System emituje cząstki w losowych pozycjach w ograniczonej przestrzeni (np. kuli); ustawienia emisji i czasu życia cząstek wpływają na całościowe zachowanie systemu



SYSTEM CZĄSTECZEK, C.D.

- Dodanie systemu cząstek:
 - GameObject > Create > Particle System lub
 - Add component > Effects > Particle System
- Na scenie panel Particle Effect



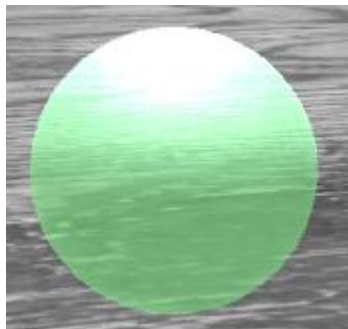
- Okno inspektora ma bardzo wiele sekcji/modułów
- Wiele własności całego systemu/cząstek może zmieniać się w czasie:
 - Constant – wartość stała
 - Curve – krzywa
 - Random between two constants – wartość losowa między 2 stałymi
 - Random between two curves – wartość losowa między 2 krzywymi
- Własność Color oraz Color over lifetime:
 - Gradient
 - Random between two gradients



SYSTEM CZĄSTECZEK – PRZYKŁAD, C.D.

- Skrypt sterujący pracą systemu cząstek, dowiązany do otaczającego go obiektu

```
void ExplodeFunction(){  
    ParticleSystem system = this.GetComponentInChildren<ParticleSystem>();  
    system.Play(); // uruchomienie systemu cząstek  
    Destroy(gameObject, system.duration); // usunięcie systemu cząstek po zdef. czasie  
}  
  
void OnTriggerEnter(Collider other){  
    if (other.gameObject.tag == "Player") {  
        ExplodeFunction ();  
    }  
}
```

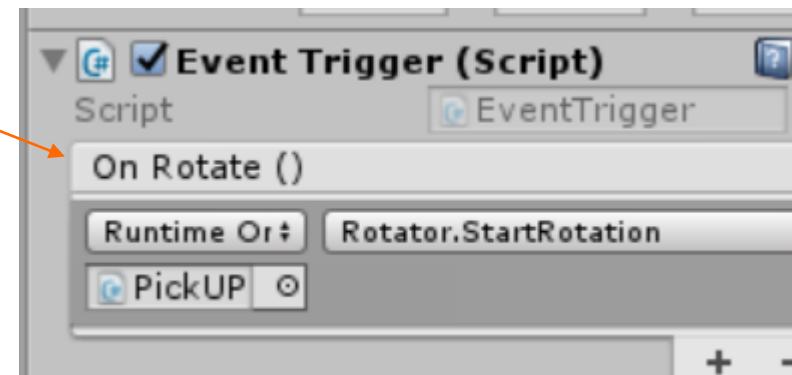


UNITY EVENTS

- Zdarzenie – wiadomość wysłana przez obiekt w celu zasygnalizowania wystąpienia pewnej akcji, np. użytkownika
- Mechanizm (wykorzystujący delegaty) do definicji funkcji callbackowych – handlerów obsługujących zdarzenia (wykorzystuje wzorzec obserwatora)
- Funkcja może być ustawiona „na stałe” w oknie Inspektora

```
using UnityEngine.Events;
```

```
public class EventTrigger : MonoBehaviour {  
    // zdarzenie powiązane z bezparametrowym delegatem typu void  
    public UnityEvent OnRotate = new UnityEvent();  
  
    void Update () {  
        if (Input.GetKeyDown("s"))  
        {  
            // wzbudzenie zdarzenia  
            OnRotate.Invoke();  
        }  
    }  
}
```



UNITY EVENTS, CONT.

- Funkcję obsługi można ustawiać w kodzie, dzięki mechanizmom rejestracji/wyrejestrowywania
- Zdarzenia z parametrami wymagają wprowadzenia skonkretyzowanej klasy pochodnej po UnityEvent

```
[Serializable]
public class MyFloatEvent : UnityEvent<float>
{
    // zdarzenie powiązane z delegatem z 1 parametrem
    public MyFloatEvent OnMove = new MyFloatEvent();

    void Update() {
        float translation = Input.GetAxis("Vertical");
        if (translation != 0)
        {
            OnMove.Invoke(translation);
        }
    }
}
```



UNITY EVENTS, CONT.

```
public void RegisterListener(UnityAction<float> call)
{
    OnMove.AddListener(call);
}
```

```
public void UnregisterListener(UnityAction<float> call)
{
    OnMove.RemoveListener(call);
}
```

Rejestracja/
Wyrejestrowywanie
Handlera obsługi

```
public class Mover : MonoBehaviour {
    private EventTrigger eventTrigger;

    private void Awake() {
        eventTrigger = FindObjectOfType<EventTrigger>();
    }

    private void OnEnable() {
        eventTrigger.RegisterListener(Move);
    }

    private void OnDisable() {
        eventTrigger.UnregisterListener(Move);
    }

    public void Move(float distance) {
        transform.Translate(0, distance, 0, Space.World);
    }
}
```

UNITY EVENTS, CONT.

- Implementacja wygodnego mechanizmu do rozgłaszania komunikatów, oparta o wzorzec obserwator (nakładka na standardowy mechanizm zdarzeń Unity)
- [Unity-Scripts/Messenger.cs at master · hybrid1969/Unity-Scripts · GitHub](#)



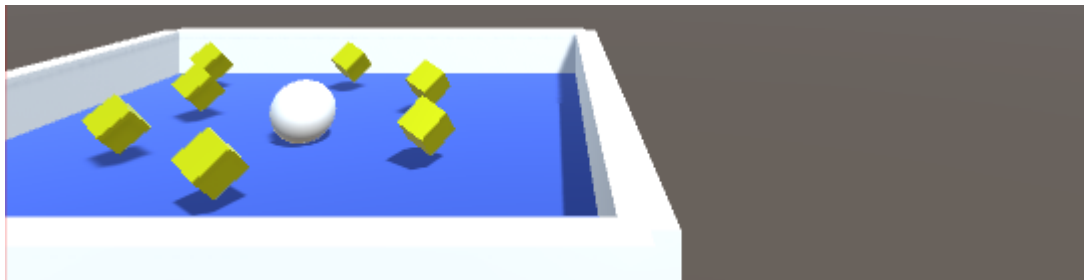
UNITY EVENTS, CONT.

- **Definicja metody wywoływanej przez słuchacza**
`void OnSpeedChanged(float speed) { ... }`
- **Rejestracja na zdarzenia**
`Messenger<float>.AddListener("event_name",
OnSpeedChanged);`
- **Wyrejestrowywanie**
`Messenger<float>.RemoveListener("event_name",
OnSpeedChanged);`
- **Wysyłanie wiadomości/Generacja zdarzenia (zamiast Invoke)**
`Messenger<float>.Broadcast("event_name", speed);`



PIERWSZA GRA 3D – PERSPEKTYWA 3CIEJ OSOBY

- Sterowanie postacią za pomocą klawiatury (ruch w lewo i w prawo, przód i tył)
- Celem gry jest zebranie obiektów, rozmieszczonych w różnych miejscach na scenie; program wyświetla liczbę zebranych obiektów
- Gracz imitowany przez sferę (bryła sztywna)
- Kamera podąża za graczem - skrypt
- Wyświetlamy informacje o liczbie zebranych obiektów (wykorzystanie systemu wiadomości)
- Wykorzystanie prostych materiałów (tylko kolor)



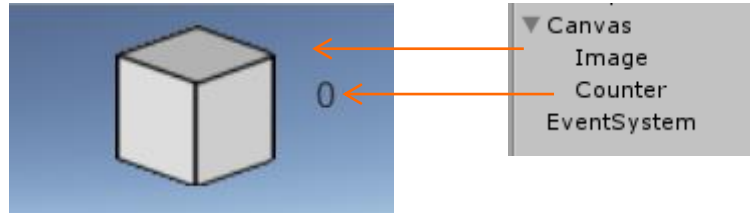
KAMERA PODĄŻA ZA GRACZEM Z PEWNEJ ODLEGŁOŚCI (ODDALONA O STAŁY WEKTOR)

○ Sterowanie ruchem kamery

```
public class CameraController : MonoBehaviour {  
    public Rigidbody player;  
    private Vector3 offset;  
  
    void Start () {  
        offset = transform.position - player.position;  
    }  
  
    // aby zapewnić, że postać gracza już się wyrysowała  
    void LateUpdate () {  
        transform.position = player.position + offset;  
    }  
}
```



UNITY EVENTS, WERSJA ROZSZERZONA



○ Reprezentacja zdarzeń (stałe)

```
public class GameEvent {  
    public const string PICK_UP = "PICK_UP";  
}
```



UNITY EVENTS, WERSJA ROZSZERZONA

○ Obsługa zdarzenia

```
public class UIController : MonoBehaviour {
    private Text scoreLabel;
    private int score = 0;

    void Awake(){
        Messenger.AddListener (GameEvent.PICK_UP, OnPickUp);
    }

    void OnDestroy(){
        Messenger.RemoveListener (GameEvent.PICK_UP, OnPickUp);
    }

    void Start(){
        scoreLabel = this.GetComponentInChildren<Text>() [0];
        scoreLabel.text = score.ToString ();
    }

    private void OnPickUp(){
        score++;
        scoreLabel.text = score.ToString ();
    }
}
```

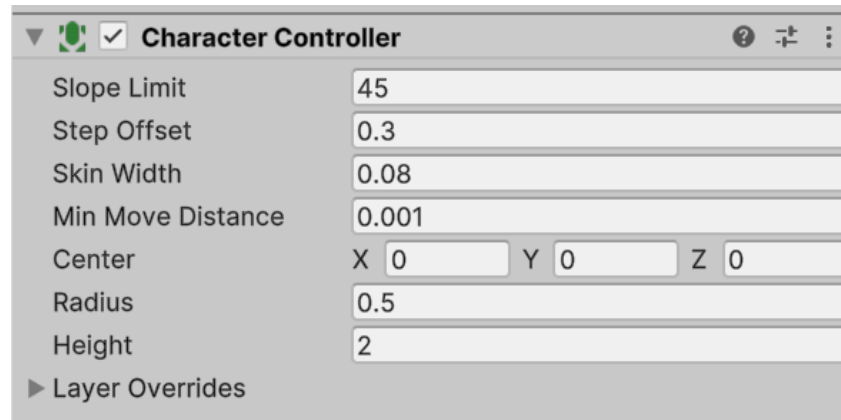
○ Wysłanie zdarzenia

```
void OnTriggerEnter(Collider other){
    if (other.gameObject.CompareTag ("PickUp")) {
        other.gameObject.SetActive (false);
        count++;
        Messenger.Broadcast (GameEvent.PICK_UP);
    }
}
```



FPC - STEROWANIE GRACZEM W ŚWIECIE 3D

- Sterowanie graczem w świecie 3D:
 - Wykorzystanie predefiniowanego komponentu (Character Controller) i wywołań funkcji Move
- Physics > Character Controller
 - Ruch postaci jest ograniczony przez możliwe kolizje (postać nie musi być bryłą sztywną)
 - Na postać nie działają siły; można nią poruszać za pomocą skryptu przez wywołanie funkcji Move



PORUSZANIE POSTACIĄ – WYWOŁANIE MOVE

```
public class KeyboardController : MonoBehaviour {
    public float speed = 15.0f;
    public float gravity = -9.8f;

    private CharacterController charController;

    void Start () {
        charController = this.GetComponent<CharacterController> ();
    }

    void Update () {
        float deltaX = Input.GetAxis ("Horizontal") * speed;
        float deltaZ = Input.GetAxis ("Vertical") * speed;

        Vector3 movement = new Vector3 (deltaX, 0, deltaZ);

        // ogranicz długość wektora do speed
        movement = Vector3.ClampMagnitude (movement, speed);

        // movement.y = 0;
        movement.y = gravity;

        // szybkość ruchu niezależna od prędkości komputera
        movement *= Time.deltaTime;
        // transformuj wektor movement do przestrzeni globalnej
        movement = transform.TransformDirection(movement);

        charController.Move (movement);    // funkcja Move komponentu CharacterController
    }
}
```

Przykładowy skrypt: Unity - Scripting API:
CharacterController.Move (unity3d.com)

PORUSZANIE POSTACIĄ – OBSŁUGA MYSZY (W DWÓCH PŁASZCZYZNACH)

```
public class MouseController : MonoBehaviour {  
    public float speed = 9.0f;        // prędkość obrotów  
  
    void Update () {  
        // Rotacja kapsuły (gracza) względem osi Y  
        transform.Rotate(0, Input.GetAxis("Mouse X") * speed, 0);  
    }  
}
```



PORUSZANIE POSTACIĄ – OBSŁUGA MYSZY (W DWÓCH PŁASZCZYZNACH)

```
public class CameraController : MonoBehaviour {  
    public float rotationSpeed = 9.0f;  
    public float minVert = -20.0f; // zakres obrotów pionowych  
    public float maxVert = 20.0f;  
  
    private float rotationX = 0; // początkowy obrót względem Y  
  
    void Update () {  
        float mY = Input.GetAxis ("Mouse Y");  
  
        // o ile zmienić X (nie możemy przekroczyć 360)  
        rotationX -= mY * rotationSpeed;  
  
        // obetnij zmianę w zakresie min-max  
        rotationX = Mathf.Clamp (rotationX, minVert, maxVert);  
  
        // rotacja kamery względem rodzica (player)  
        transform.localEulerAngles = new Vector3 (rotationX, transform.localEulerAngles.y, 0);  
    }  
}
```

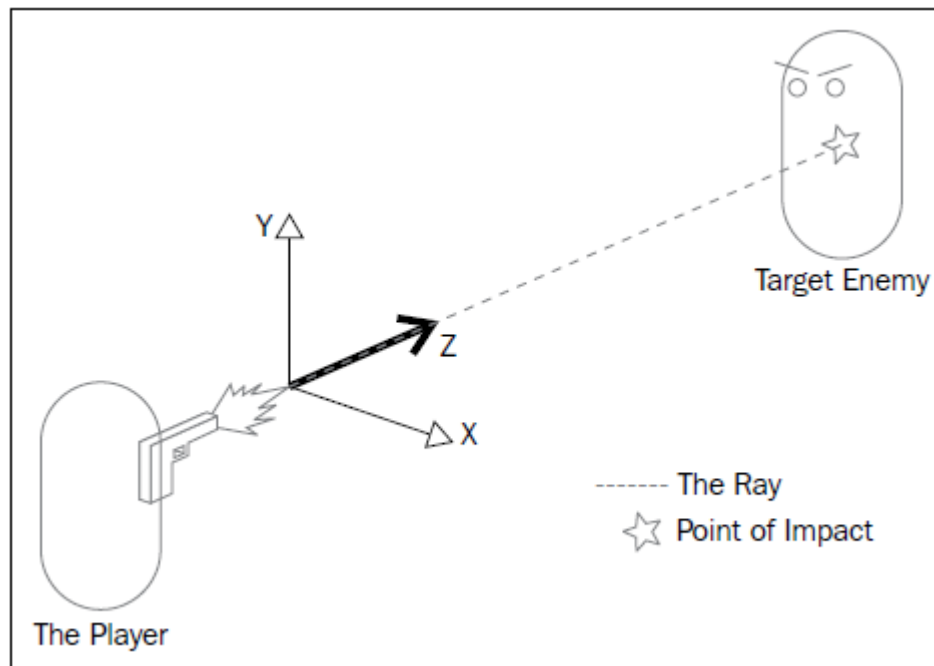
GRA 3D – STRZELANIE DO OBIEKTÓW

- Gra z perspektywy pierwszej osoby
- Gracz widzi fragment broni (i celownik)
- Strzelanie z broni: dźwięk, system cząsteczek (efekt wystrzału)



RZUT PROMIENIA (RAYCASTING)

- Mechanizm „przewidywania” kolizji
- Rozwiązuje problem pomijania elementów, które pocisk minął między ramkami



RZUT PROMIENIA (RAYCASTING), C.D.

- Klasa Physics:

```
public static bool Raycast(  
Vector3 origin, Vector3 direction,  
out RaycastHit hitInfo,  
float maxDistance = Mathf.Infinity,  
int layerMask = DefaultRaycastLayers, ...);
```

```
public static bool Raycast(  
Ray ray, out RaycastHit hitInfo,  
float maxDistance = Mathf.Infinity,  
int layerMask = DefaultRaycastLayers, _...);
```



RZUT PROMIENIA (RAYCASTING), C.D.

- Przykład wykorzystania:

```
public class RaycastExample : MonoBehaviour
{
    public Missile missile;

    void FixedUpdate()
    {
        RaycastHit hit;

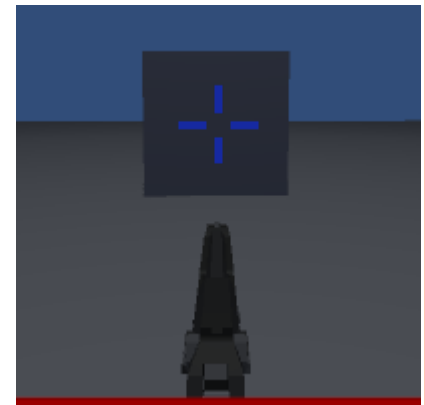
        if (Physics.Raycast(transform.position, -Vector3.up, out hit))
            print("Found an object - distance: " + hit.distance);
    }
}
```



DODANIE CELOWNIKA, C.D.

○ Skrypt pistoletu:

```
public Texture2D viewfinderTexture;  
private Rect viewfinderPos;  
  
// Use this for initialization  
void Start () {  
    Cursor.visible = false; // hide mouse cursor  
    audioSource = GetComponent<AudioSource>();  
    audioSource.clip = pistolShotClip;  
    viewfinderPos = new Rect ((Screen.width - viewfinderTexture.width) / 2,  
        (Screen.height - viewfinderTexture.height) / 2,  
        viewfinderTexture.width, viewfinderTexture.height);  
}  
  
void OnGUI(){  
    GUI.DrawTexture (viewfinderPos, viewfinderTexture);  
}
```



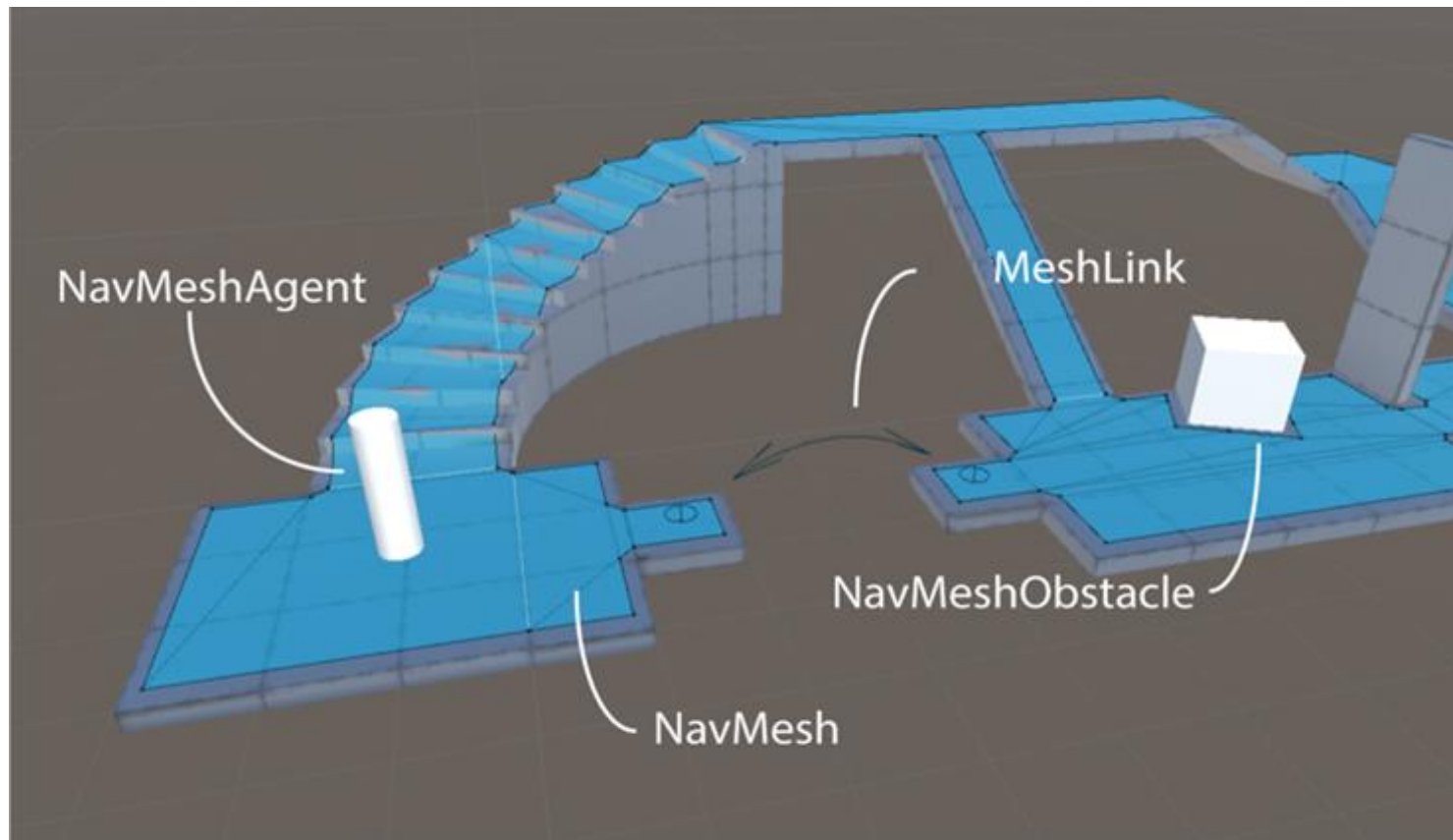
ROZWIĄZYWANIE PROBLEMU PRZENIKANIA BRONI PRZEZ ŚCIANY

- Dodać na scenę drugą kamerę (np. jako dziecko kamery głównej), która będzie wyświetlać tylko Pistolet (umieszczony na jednej z warstw);
- Parametry dodatkowej kamery:
 - Clear flags: Depth only,
 - Depth 1 (więcej niż dla kamery głównej)



AI – ZNAJDOWANIE ŚCIEŻEK I NAWIGACJA

- Główne elementy systemu
(Window > AI > Navigation)



AI – ZNAJDOWANIE ŚCIEŻEK I NAWIGACJA, C.D.

- Powierzchnia siatki (NavMeshSurface) – komponent w Unity, który służy do wypiekania (bake) siatki nawigacyjnej
- Siatka nawigacyjna (Navigation Mesh, NavMesh) – struktura danych opisująca powierzchnię, zbudowana z wielokątów
- Agent nawigacyjny (NavMesh Agent) – komponent dodawany do postaci, które dążą do wskazanego celu (A^*) po siatce nawigacyjnej; wiedzą jak się unikać oraz jak unikać poruszających się przeszkód
- Łącznik między siatkami (Off-Mesh Link) – komponent pozwalający na utworzenie skrótu nawigacyjnego (np. skok w górę, skok w bok)
- Przeszkoda (NavMesh Obstacle) – komponent dodawany do (poruszających się) obiektów, które powinny być omijane przez agentów



AI – ZNAJDOWANIE ŚCIEŻEK I NAWIGACJA

- Główne elementy systemu
(Window > AI > Navigation)

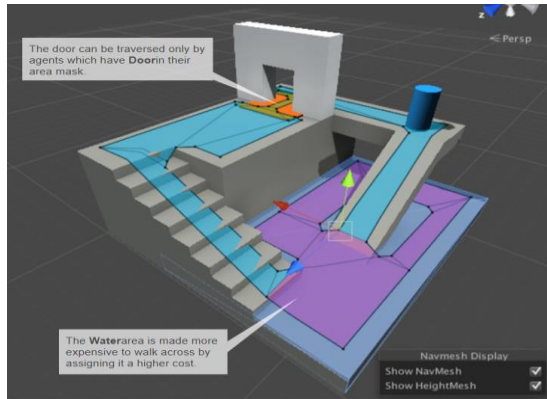
Komponent	Funkcja	Dodaj na...
NavMeshSurface	Tworzy siatkę nawigacji	Podłogi, schody, platformy
NavMeshAgent	Pozwala AI się poruszać	NPC, przeciwnicy
NavMeshLink	Łączy różne NavMeshe	Przejścia (schody, mosty)
NavMeshObstacle	Blokuje AI	Skrzynie, drzwi, bariery



AI – ZNAJDOWANIE ŚCIEŻEK I NAWIGACJA, C.D.

- Konfiguracja systemu: koszt przejścia przez powierzchnię, nie wpływa na prędkość, tylko na preferencję wyboru trasy
- Istnieje możliwość zmiany kosztu w trakcie gry:

```
public void SetAreaCost  
(int areaIndex, float areaCost);
```



Navigation		
Navigation		
Agents Areas		
	Name	Cost
Built-in 0	Walkable	1
Built-in 1	Not Walkable	1
Built-in 2	Jump	2
User 3	Water	3
User 4		1
User 5		1
User 6		1
User 7		1
User 8		1
User 9		1
User 10		1
User 11		1
User 12		1

AI – ZNAJDOWANIE ŚCIEŻEK I NAWIGACJA, C.D.

- Modyfikacja NavMesh w kontekście nawigacji agentów AI; wymaga ponownego wypiekania

Cecha	NavMeshModifier	NavMeshModifierVolume
Działa na	Całym obiekcie (np. podłoga, most)	Tylko w określonym obszarze (np. fragment terenu, błoto, woda)
Kształt	Dopasowany do obiektu	Możesz ustawić własny rozmiar (<code>size</code>)
Wykluczanie z NavMesh	Możesz usunąć obiekt z NavMesh	Nie usuwa obiektów, tylko zmienia zachowanie agentów
Zmiana kosztu ruchu AI	Nie	Tak (np. błoto spowalnia AI)
Ograniczenie do konkretnych agentów	Tak	Tak



AI – ZNAJDOWANIE ŚCIEŻEK I NAWIGACJA, C.D.

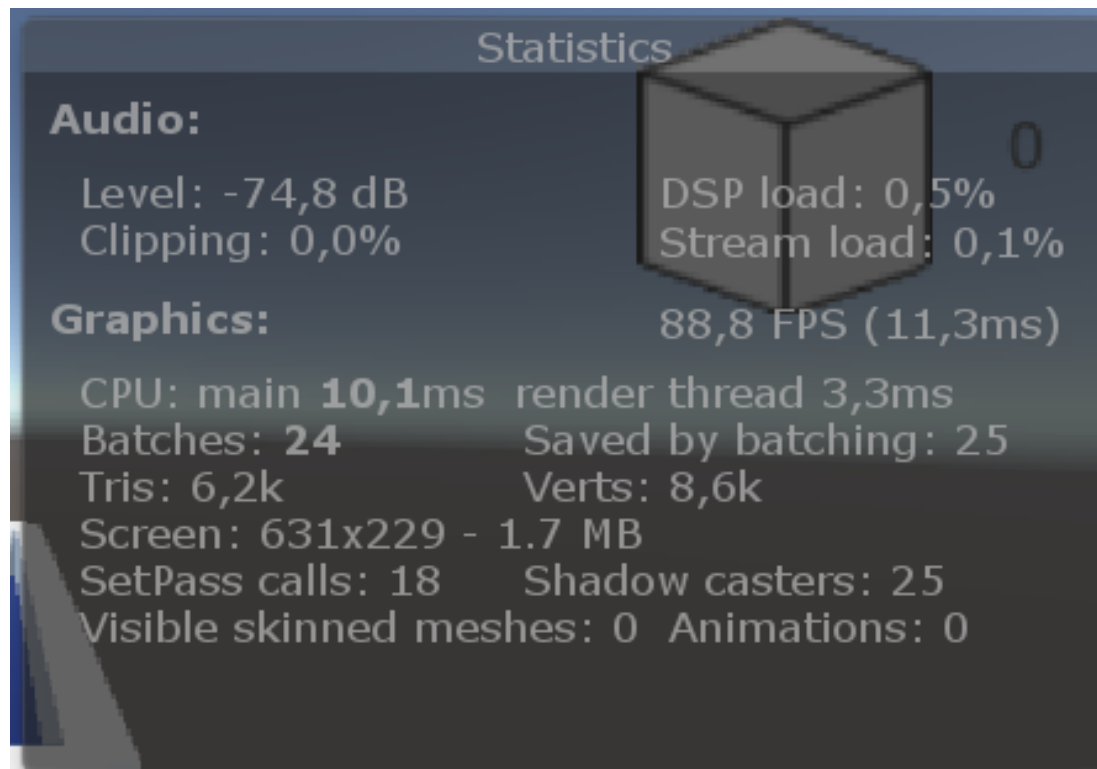
- Przykładowy skrypt dołączany do agenta nawigacyjnego, który pozwala zdefiniować jego cel

```
using UnityEngine;
using UnityEngine.AI;
public class MoveTo: MonoBehaviour {
    public Transform goal;
    void Start () {
        NavMeshAgent agent =
            GetComponent<NavMeshAgent>();
        agent.destination = goal.position;
    }
}
```



OKNO STATYSTYK

- Game > Stats



PYTANIA KONTROLNE

- Wymień rodzaje źródeł światła czasu-rzeczywistego dostępne w Unity
- Zdefiniuj pojęcia: tekstura, materiał, zacięniacz; jakie są między nimi relacje?
- Czym jest ciasteczko (cookie)?
- Czym jest Skybox
- Czym jest system cząsteczek? Podaj sytuację, gdy może mieć zastosowanie
- Czym jest Raycasting? Do czego może mieć zastosowanie?
- Wymień główne elementy wbudowanego systemu nawigacji AI

